

CHARUSAT Online Course Assistant

Project Review Notes — everything in 30 minutes

1. What is this project

A **RAG (Retrieval-Augmented Generation) chatbot** that answers questions about CHARUSAT's online degree programmes — **BBA, BCA, MBA, MCA** (fees, eligibility, duration, curriculum, admissions, exams, refunds). It retrieves relevant chunks from a vector database and generates an answer using an LLM.

Stack (one line): Python **FastAPI** + **LangChain** backend · **ChromaDB** vector store · **Groq/Gemini** LLMs · **MongoDB Atlas** for users + chat history · **React 19** + **Vite** + **Tailwind v4** frontend.

2. Repository structure — what each part is for

Path	What it is / why it is in the project
requirements.txt (root)	LEGACY duplicate of backend/requirements.txt (adds jupyter, crawl4ai, sentence-transformers). Not used by deployment — removable.
.env.example / .env.local / .vercelignore / render.yaml / vercel.json	Environment templates + deploy configs: Render runs the backend, Vercel hosts the static frontend (SPA rewrites).
BACKEND_DIFF_MAIN_V S_DEPLOY.md, walkthrough.md, CHANGES.txt, changes_an d_outputs_summary.txt, co mpleted-work-summary.txt, DEPLOYMENT_TROUBLE SHOOTING_LOG.txt, today-progress.md	Developer notes / work logs / smoke-test report. Useful for review, but they are docs, not code.
CHARUSAT_Chatbot_Pres entation_Guide.pdf, Online program questions.docx	Review reference material (presentation guide + sample questions).
backend/	FastAPI RAG backend (see section 3). Core project.
frontend/	React SPA frontend (see section 4). Core project.
deployment/	OLD Vercel working folder — only node_modules remains, no source. Unnecessary now that frontend/ is used.
backend/.venv, frontend/node_modules, __pycache__	Local tooling only — gitignored, not part of the repo.

3. Backend — FastAPI (port 8000)

Entry: backend/api.py — app version 2.0.0. On startup it loads ChromaDB (rebuilds from knowledge_base if missing), connects MongoDB (best-effort), seeds dev admin, and mounts 4 routers. All original endpoints are kept for backward compatibility.

File (backend/app/...)	Role / key facts
config.py	All constants: embedding model, TOP_K=6, CANDIDATE_POOL=20, chunk sizes (600 md / 800 pdf / 3600 table), LLM temperature/timeouts, attribute taxonomy, source-authority tiers, MIN_RELEVANCE 0.52, CONTEXT_TOTAL 5600 chars.
embeddings.py	FastEmbed (ONNX) wrapper for BAAI/bge-small-en-v1.5, 384-dim, ~100MB RAM. Falls back to HuggingFace/other model if unavailable. EmbeddingStore manages Chroma.
retriever.py	RAGRetriever: detects intent (programme / overview / general), gathers a wide candidate pool per programme, scores with keyword + phrase + source-authority bonuses, dedupes and diversifies, picks top-K. Forces one chunk per programme for cross-programme questions.
rag_chain.py	Pipeline: retrieve → conflict detection → build context (bounded char budget) → generate answer → sanitize. Tracks timing + status.
llm.py	LLMClient. Primary = Groq (openai/gpt-oss-120b, failover: gpt-oss-20b, qwen3.8-27b, allam-2-7b). Gemini models configured but Gemini key currently off (repo only needs GROQ_API_KEY). Deterministic local fallback answers from evidence; strips internal source refs from answers.
prompt.py, conflict.py, deduplicate.py, source_authority.py, scraper.py	Prompt builder · claim extraction + conflict notices (scoped to the attribute asked) · Jaccard near-duplicate removal · official-source priority bonus · crawls charusat.online into knowledge_base.
ingestion.py, pdf_ingestion.py	Markdown KB → chunks with metadata; PDFIngestor extracts text + compact tables via PyMuPDF (table chunk 3600 chars so fee tables stay whole).
database.py	PyMongo sync client. DB charusat_assistant; collections users, conversations, messages; indexes; graceful None fallback if Mongo absent.
auth.py	bcrypt hashing (72-byte truncation), JWT HS256 (1-week expiry), deps get_current_user / get_optional_user / require_admin, Google OAuth token verify, dev auto-login (DEV_AUTO_LOGIN=true).

File (backend/app/...)	Role / key facts
models.py	Data-access layer (CRUD) for users / conversations / messages; auto conversation titles; cascade delete of messages.
routes/	auth_routes (signup/login/logout/me/google/dev-login), chat_routes (/api/chat + /api/chat/migrate + small-talk replies + suggested follow-ups), conversation_routes (list/create/get/rename/delete), admin_routes (/api/admin/stats).
backend/vector_db/	ChromaDB data (SQLite + colection UUID 513ebb91..., rebuilt today; old UUID 3654ea0a... deleted). source_manifest.json tracks KB state for rebuilds.
backend/knowledge_base/	Course content: 13 scraped .md pages + programs/ (4 programme pages + contact) + bba/bca-admission-data.md (new today) + pdfs/ (PPR BBA/BCA/MBA/MCA + Fees Refund Policy).

4. Frontend — React 19 + Vite (port 5173)

Tech: React 19, react-router-dom 7, Tailwind v4 via @tailwindcss/vite (no tailwind.config.js), custom fetch client (no axios). Dev proxy: /api → http://localhost:8000.

Frontend part	What it does
src/pages/	Landing (marketing home) · Login / Signup · Chat (main assistant, guest + logged-in, ?q= presets, /chat/:conversationId) · Settings (account, protected) · Admin (stats, admin-only).
src/contexts/	AuthContext (user, JWT, guest-limit 4 via sessionStorage, dev auto-login, Google) · ThemeContext (dark/light/system, persisted).
src/services/	api.js (base url http://localhost:8000, bearer-token fetch wrapper) · auth.js · chat.js (conversations CRUD + /api/chat).
src/components/chat/	ChatLayout, Sidebar (history/search/rename/delete), MessageList, ChatComposer, AssistantMessage (sources, feedback, follow-ups), UserMessage, WelcomeScreen, GuestLimitModal, SuggestedQuestions.
src/components/	auth/ProtectedRoute + GoogleButton · ui/ThemeToggle · site/SiteNav + SiteFooter · RichText (markdown-lite renderer).
src/lib/	Empty leftover folder (old api.js replaced by src/services) — removable.

5. Database, Authentication, Vector DB

Area	Details
Database	MongoDB Atlas (db charusat_assistant). users {name, email, password_hash, google_id, avatar, role}, conversations {user_id, title, timestamps}, messages {conversation_id, role, content, sources, suggested_questions}. Not needed to run RAG — graceful degradation.
Authentication	Email/password (bcrypt) or Google OAuth (optional). JWT (HS256, 7 days) stored in localStorage; ProtectedRoute guards /settings and /admin; guest mode up to 4 messages then must sign up; guest history migrates on login.
Vector DB	ChromaDB persisted at backend/vector_db. Collection 'langchain'. Embedding BAAI/bge-small-en-v1.5 (FastEmbed ONNX, 384-dim). Today: wiped & rebuilt → 389 chunks, 0 duplicates (new UUID), MD + PDFs re-ingested.

6. Ports & commands to run the project

Step	Command	Port / note
Backend	cd backend; uvicorn api:app --reload --port 8000	8000 · interactive API docs at /docs
Frontend	cd frontend; npm install; npm run dev	5173 · proxies /api to :8000
Production build	cd frontend; npm run build; npm run preview	dist/ output
Rebuild vector DB	cd backend; python rebuild_vectordb.py	rebuilds Chroma from knowledge_base
Ingest PDFs	cd backend; python ingest_pdfs.py	adds PDFs to vector store
CLI mode (no web)	cd backend; python main.py	terminal chatbot
Smoke test	python backend/smoke_test_backend.py	9 checks (used earlier)
Deploy backend	Render: build.sh, start uvicorn api:app --host 0.0.0.0 --port \$PORT	uses branch deploy / rootDir backend
Deploy frontend	Vercel: vercel.json → npm run build, dist	serverless static

Required env vars (backend/.env): GROQ_API_KEY (live answers) · optional: GOOGLE_API_KEY (Gemini tier), MONGODB_URI, JWT_SECRET, ADMIN_EMAIL, DEV_AUTO_LOGIN, GUEST_MESSAGE_LIMIT, GOOGLE_CLIENT_ID. Frontend (.env): VITE_API_URL, VITE_GOOGLE_CLIENT_ID, VITE_GUEST_MESSAGE_LIMIT.

7. What was done today (root-cause fixes + frontend polish)

Root causes of earlier bad answers (notes in CHATB/tmp):

- **Groq models were decommissioned** — old names (llama-3.3-70b, mixtral, gemma2) returned errors, so every answer fell to the weak local fallback. Fixed: switched to live models **gpt-oss-120b** / **gpt-oss-20b** / **qwen3.8-27b** / **allam-2-7b** (llm.py + config.py).
- **Fee tables were shredded** — the fee table extracted to 6,359 chars and was truncated before the numbers, and fragments tripped the diversity cap. Fixed: compact table extraction (6,359 → 3,312 chars), PDF_TABLE_CHUNK_SIZE 1200 → **3600** (whole table = 1 chunk), CONTEXT_TABLE_MAX_CHARS 3400, total context 4200 → 5600, TOP_K 4 → 6.

- **Conflicting fees** — conflict notices fired on any attribute; “total credits 133 vs 130” polluted fee answers. Fixed: conflicts now scoped to the attribute family actually asked (added cost→fee family).
- **Stale/duplicated vector DB** — a running server held chroma.sqlite3 open, so a rebuild appended duplicates (201 dup chunks). Fixed: stopped server, wiped + rebuilt Chroma → **389 chunks, 0 dupes**, new collection UUID, PDFs re-ingested, server restarted on :8000.

Verified end-to-end (Groq gpt-oss-120b): BBA fee = 50,000/yr × 3 = 1,50,000 total (+ exam-fee nuance) · semesters per programme (BCA 6, BBA 6, MBA 4, MCA not stated) · total cost Online BBA = 1,50,000 INR · refund policy points to Fees Refund Policy.

Also in the working tree (uncommitted): 13-site re-scrape, new bba/bca-admission-data.md, rebuilt vector store (389 chunks), plus a large frontend rebuild commit (staged: professional React+Tailwind chat UI with auth, conversations, admin, theme). Main branch is currently 20 commits ahead of origin/main.

8. API endpoints at a glance

Method / Path	Auth	Purpose
POST /chat · GET /health · GET /programs · GET /sources · POST /refresh	no	Legacy stateless RAG endpoints (kept for compatibility).
POST /api/auth/signup login logout me google dev-login	me = Bearer	Registration, login, profile, Google OAuth, dev-only auto-login.
POST /api/chat POST /api/chat/migrate	optional / yes	Stateful chat (saves to Mongo when logged in, guest otherwise); migrate guest chat after signup.
GET/POST /api/conversations · GET/PATCH/DELETE /api/conversations/{id}	yes	Conversation thread CRUD + history.
GET /api/admin/stats	admin	Users, conversations, DB & RAG status.

9. Unnecessary / cruft files in the repo (main branch)

Safe to delete — not used by code or deployment:

- **requirements.txt (root)** — old duplicate of backend/requirements.txt (has jupyter, crawl4ai, etc.). Deployment uses backend/requirements.txt.
- **deployment/ folder** — empty legacy Vercel workspace (only node_modules). The real frontend lives in frontend/.
- **Stray note/log files:** CHANGES.txt, changes_and_outputs_summary.txt, completed-work-summary.txt, DEPLOYMENT_TROUBLESHOOTING_LOG.txt, walkthrough.md, today-progress.md, backend/CHAT_HISTORY_CHANGES.txt, backend/changes_and_outputs_summary.txt — work logs from earlier sessions; keep in a notes/ folder or delete before review.
- **frontend/src/lib/** — empty folder left after moving api.js into src/services/.

Already cleaned up (gone from git): old Chroma collection 3654ea0a (replaced by rebuilt 513ebb91) · old legacy frontend components (Hero, FAQ, Footer, ChatPanel, Programs...).

Not committed, keep out of the repo: backend/.env, .env.local, .venv/, node_modules/, __pycache__ — already gitignored. Don't push real API keys.

10. Quick-revision facts

Fact	Value
App versions	Backend FastAPI 2.0.0 · Frontend 1.0.0
LLM / fallback	Groq gpt-oss-120b (primary) → deterministic local answer (no API)
Embeddings	BAAI/bge-small-en-v1.5, 384-dim, FastEmbed ONNX (~100 MB RAM)
Vector store	ChromaDB, 389 chunks (today), collection 'langchain'
Vector DB path	backend/vector_db
LLM context budget	6 chunks, ~5,600 chars max
Live knowledge sources	13 website pages + 5 PDFs (programme PPRs + refund policy) + 2 admission-data pages

Review tip: run backend on :8000 and frontend on :5173, then demo a fee question (proves RAG + Groq), sign up (proves Mongo auth), and open Admin (proves roles).